

api.slmstudio.ai/v1
smart-model-tune.lovable.app

Version	1.0
Date	2026-05-04
Backend	Supabase (PostgreSQL + Auth)
Auth	Supabase JWT API Key sk-slm-*
Frontend	React 18, TypeScript, Vite, Tailwind CSS

Table of Contents

1. Authentication
2. Projects API
3. Models API
4. Datasets API
5. Deployments API
6. API Keys API
7. Analytics API
8. Model Inference API (POST /v1/chat/completions)
9. Database Schema

1. Authentication

1.1 Supabase Auth (Web Application)

Method	Supabase JWT – Email / Password
Token Location	HTTP Authorization header + localStorage (browser)
Token Format	Bearer <supabase-jwt>
Token Lifetime	Auto-refreshed (autoRefreshToken: true)
Sign Up	supabase.auth.signUp({ email, password })
Sign In	supabase.auth.signInWithPassword({ email, password })
Sign Out	supabase.auth.signOut()

1.2 API Keys (Model Inference)

Format	sk-slm-{name4chars}-{suffix4chars} (e.g. sk-slm-prod-a8f3)
Header	Authorization: Bearer sk-slm-xxxx-yyyy
Scope	Authenticates calls to api.slmstudio.ai/v1
Storage	Stored in Supabase table api_keys (prefix + suffix only)
Visibility	Full key shown only at creation time

1.3 Row-Level Security (RLS)

Enforcement	All Supabase tables have RLS enabled
Filter	USING (user_id = auth.uid())
Impact	Users can only read / write their own rows – no cross-tenant access

2. Projects API

File: `src/lib/projectsApi.ts` | Supabase table: `projects` | RLS: `user_id = auth.uid()`

Function	Operation	Parameters	Returns
<code>listProjects()</code>	SELECT ORDER BY updated_at DESC	none	Promise<Project[]>
<code>getProject(id)</code>	SELECT WHERE id = :id	id: string (UUID)	Promise<Project null>
<code>createProject(input)</code>	INSERT	name: string * description?: string taskType: TaskType * baseModel: BaseModel * epochs?: number (def: 3) learningRate?: number (def: 0.0002) datasetSize?: number (def: 0) tags?: string[]	Promise<Project>
<code>updateProject(id, patch)</code>	UPDATE WHERE id = :id	id: string (UUID) patch: Partial<Project> (name, description, taskType, baseModel, status, progress, epochs, learningRate, datasetSize, creditsCost, pinned, tags)	Promise<Project>
<code>deleteProject(id)</code>	DELETE WHERE id = :id	id: string (UUID)	Promise<void>

Enum Values

TaskType	classification ner qa function-calling extraction ranking
BaseModel	qwen2.5-1.5b qwen2.5-3b gemma-2-2b phi-3-mini llama-3.2-1b smollm2-1.7b
ProjectStatus	training completed failed queued paused

Project Response Object

Field	Type	Description
id	string	UUID primary key
name	string	Project display name
description	string	Project description
taskType	TaskType	Fine-tuning task type
baseModel	BaseModel	Pre-trained model used
status	string	Current project status
progress	number	Training progress 0–100
epochs	number	Number of training epochs
learningRate	number	Optimizer learning rate

Field	Type	Description
datasetSize	number	Number of training samples
creditsCost	number	Credits consumed
pinned	boolean	Whether project is pinned in UI
tags	string[]	User-defined labels
createdAt	string	ISO 8601 timestamp
updatedAt	string	ISO 8601 timestamp

3. Models API

File: `src/lib/modelsApi.ts` | Supabase table: `trained_models` | RLS: `user_id = auth.uid()`

Function	Operation	Parameters	Returns
<code>listModels()</code>	<code>SELECT ORDER BY created_at DESC</code>	none	<code>Promise<TrainedModelExt[]></code>
<code>getModel(id)</code>	<code>SELECT WHERE id = :id</code>	<code>id: string (UUID)</code>	<code>Promise<TrainedModelExt null></code>

TrainedModelExt Response Object

Field	Type	Description
<code>id</code>	<code>string</code>	UUID primary key
<code>name</code>	<code>string</code>	Model display name
<code>projectId</code>	<code>string</code>	FK to projects table
<code>baseModel</code>	<code>BaseModel</code>	Base model used for fine-tuning
<code>taskType</code>	<code>TaskType</code>	Task the model was trained for
<code>accuracy</code>	<code>number</code>	Accuracy score (0.0 – 1.0)
<code>f1Score</code>	<code>number</code>	F1 score (0.0 – 1.0)
<code>precision</code>	<code>number</code>	Precision score (0.0 – 1.0)
<code>recall</code>	<code>number</code>	Recall score (0.0 – 1.0)
<code>latencyMs</code>	<code>number</code>	Average inference latency (ms)
<code>fileSize</code>	<code>string</code>	File size string (e.g. '245 MB')
<code>format</code>	<code>string</code>	File format (e.g. 'gguf')
<code>status</code>	<code>string</code>	<code>ready</code> <code>deploying</code> <code>deployed</code>
<code>createdAt</code>	<code>string</code>	ISO 8601 timestamp

4. Datasets API

File: `src/lib/datasetsApi.ts` | Supabase table: `datasets` | RLS: `user_id = auth.uid()`

Function	Operation	Parameters	Returns
<code>listDatasets()</code>	<code>SELECT ORDER BY created_at DESC</code>	none	<code>Promise<UserDataset[]></code>

UserDataset Response Object

Field	Type	Description
<code>id</code>	<code>string</code>	UUID primary key
<code>projectId</code>	<code>string null</code>	FK to projects table (nullable)
<code>name</code>	<code>string</code>	Dataset display name
<code>description</code>	<code>string</code>	Dataset description
<code>rows</code>	<code>number</code>	Number of data rows
<code>columns</code>	<code>number</code>	Number of columns
<code>fileSize</code>	<code>string</code>	File size string (e.g. '12.4 MB')
<code>format</code>	<code>string</code>	File format (e.g. 'jsonl')
<code>qualityScore</code>	<code>number</code>	Quality score 0.0 – 1.0
<code>createdAt</code>	<code>string</code>	ISO 8601 timestamp

5. Deployments API

File: `src/lib/deploymentsApi.ts` | Supabase table: `deployed_endpoints` | RLS: `user_id = auth.uid()`

Function	Operation	Parameters	Returns
<code>listEndpoints()</code>	SELECT ORDER BY created_at DESC	none	Promise<DeployedEndpoint[]>
<code>setEndpointStatus (id, status)</code>	UPDATE WHERE id = :id	id: string (UUID) status: 'active' 'inactive'	Promise<void>

DeployedEndpoint Response Object

Field	Type	Description
<code>id</code>	string	UUID primary key
<code>modelId</code>	string null	FK to trained_models (nullable)
<code>modelName</code>	string	Name of the deployed model
<code>projectName</code>	string	Name of the parent project
<code>endpointUrl</code>	string	Full URL of the inference endpoint
<code>status</code>	string	active inactive scaling
<code>requestsPerMin</code>	number	Current requests per minute
<code>avgLatencyMs</code>	number	Average latency in milliseconds
<code>errorRate</code>	number	Error rate as percentage (0.0 – 100.0)
<code>uptime</code>	number	Uptime percentage (0.0 – 100.0)
<code>rateLimitPerMin</code>	number	Configured rate limit per minute
<code>burstLimit</code>	number	Burst request limit

6. API Keys API

File: `src/lib/apiKeysApi.ts` | Supabase table: `api_keys` | RLS: `user_id = auth.uid()`

Function	Operation	Parameters	Returns	Notes
<code>listApiKeys()</code>	SELECT ORDER BY created_at DESC	none	<code>Promise<ApiKey[]></code>	All keys (active + revoked)
<code>createApiKey(name)</code>	INSERT	name: string Auto-generates: prefix: <code>sk-slm-{name[0:4]}</code> suffix: random 4-char	<code>Promise<ApiKey></code>	Full key visible once only
<code>revokeApiKey(id)</code>	UPDATE status='revoked'	id: string (UUID)	<code>Promise<void></code>	Record kept, not deleted

ApiKey Response Object

Field	Type	Description
<code>id</code>	string	UUID primary key
<code>name</code>	string	User-defined label for the key
<code>keyPrefix</code>	string	First part of key (e.g. <code>sk-slm-prod</code>)
<code>keySuffix</code>	string	Last 4 characters of the key
<code>status</code>	string	active revoked
<code>lastUsedAt</code>	string null	ISO 8601 timestamp of last use, or null
<code>createdAt</code>	string	ISO 8601 creation timestamp

7. Analytics API

File: `src/lib/analyticsApi.ts` | Supabase table: `api_call_events` | RLS: `user_id = auth.uid()`

Function	Type	Parameters	Returns
<code>listCallEvents(range)</code>	Supabase SELECT	range: '24h' '7d' '30d' '90d' Filters: <code>created_at >= now - N</code> days Limit: 5,000 records	<code>Promise<CallEvent[]></code>
<code>summarize(events)</code>	Pure function	events: <code>CallEvent[]</code>	<code>AnalyticsSummary { totalCalls, avgLatency, errorRate, uptime }</code>
<code>bucketByTime(events, range)</code>	Pure function	events: <code>CallEvent[]</code> range: string	<code>{ calls: BucketPoint[], latency: LatencyPoint[] }</code> Buckets: 24h=hourly, else daily Latency: p50, p95, p99
<code>endpointStats(events)</code>	Pure function	events: <code>CallEvent[]</code>	<code>EndpointStat[] { endpoint, calls, avgLatency, errorRate }</code> Sorted by calls DESC

CallEvent Object

Field	Type	Description
<code>id</code>	string	UUID of the event record
<code>endpoint</code>	string	Endpoint URL that was called
<code>statusCode</code>	number	HTTP status code (e.g. 200, 400, 500)
<code>latencyMs</code>	number	Response latency in milliseconds
<code>createdAt</code>	string	ISO 8601 timestamp

AnalyticsSummary Object

Field	Type	Description
<code>totalCalls</code>	number	Total number of API calls in the selected range
<code>avgLatency</code>	number	Average latency in milliseconds
<code>errorRate</code>	number	Percentage of calls with <code>statusCode >= 400</code>
<code>uptime</code>	number	Estimated uptime percentage ($100 - \text{errorRate} * 0.1$)

8. Model Inference API

POST https://api.slmstudio.ai/v1/chat/completions

Compatibility	OpenAI-compatible REST API – use openai SDK with base_url override
Authentication	Authorization: Bearer <api-key> (key format: sk-slm-*)
Content-Type	application/json

Request Body Parameters

Field	Type	Description
model	string *required	Model name as deployed in SLM Studio (e.g. 'thai-ner-v2')
messages	array *required	[[role: 'user' 'assistant' 'system', content: string]]
temperature	number optional	Sampling temperature 0.0 – 2.0 (default: 1.0)
max_tokens	number optional	Maximum tokens to generate in the response
stream	boolean optional	Stream response token-by-token (default: false)
top_p	number optional	Nucleus sampling parameter (default: 1.0)

Code Examples

Python (openai SDK)

```
import openai

client = openai.OpenAI(
    api_key="sk-slm-prod-a8f3",
    base_url="https://api.slmstudio.ai/v1"
)

response = client.chat.completions.create(
    model="thai-ner-v2",
    messages=[{"role": "user", "content": "■■■■■■■■■■: ■■■■■■ ■■■. ■■■■■"}],
    temperature=0.1
)
print(response.choices[0].message.content)
```

cURL

```
curl -X POST https://api.slmstudio.ai/v1/chat/completions \
-H "Authorization: Bearer sk-slm-prod-a8f3" \
-H "Content-Type: application/json" \
-d '{
  "model": "thai-ner-v2",
  "messages": [{"role": "user", "content": "■■■■■■■■■■: ■■■■■■ ■■■. ■■■■■"}],
  "temperature": 0.1
}'
```

JavaScript (fetch)

```
const response = await fetch("https://api.slmstudio.ai/v1/chat/completions", {
  method: "POST",
  headers: {
    "Authorization": "Bearer sk-slm-prod-a8f3",
    "Content-Type": "application/json",
  },
  body: JSON.stringify({
    model: "thai-ner-v2",
    messages: [{ role: "user", content: "■■■■■■■■■■: ■■■■■■ ■■■. ■■■■■■" }],
    temperature: 0.1,
  }),
});
const data = await response.json();
console.log(data.choices[0].message.content);
```

9. Database Schema (Supabase / PostgreSQL)

TABLE: profiles*RLS: SELECT own | INSERT own | UPDATE own*

Column	Type	Constraint / Notes
id	UUID	PK
user_id	UUID	FK → auth.users
display_name	TEXT	
avatar_url	TEXT	nullable
created_at	TIMESTAMPTZ	default now()
updated_at	TIMESTAMPTZ	default now()

TABLE: projects*RLS: SELECT/INSERT/UPDATE/DELETE own | idx_projects_user_id, idx_projects_updated_at*

Column	Type	Constraint / Notes
id	UUID	PK default gen_random_uuid()
user_id	UUID	FK → auth.users
name	TEXT	NOT NULL
description	TEXT	
task_type	TEXT	NOT NULL
base_model	TEXT	NOT NULL
status	TEXT	default 'queued'
progress	INTEGER	default 0 (0–100)
epochs	INTEGER	default 3
learning_rate	NUMERIC	default 0.0002
dataset_size	INTEGER	default 0
credits_cost	NUMERIC	default 0
pinned	BOOLEAN	default false
tags	TEXT[]	default '{}'
created_at	TIMESTAMPTZ	default now()
updated_at	TIMESTAMPTZ	default now()

TABLE: trained_models

RLS: SELECT/INSERT/UPDATE/DELETE own | idx_trained_models_user

Column	Type	Constraint / Notes
id	UUID	PK
user_id	UUID	FK → auth.users
project_id	UUID	FK → projects nullable
name	TEXT	
base_model	TEXT	
task_type	TEXT	
accuracy	NUMERIC	
f1_score	NUMERIC	
precision	NUMERIC	
recall	NUMERIC	
latency_ms	INTEGER	
file_size	TEXT	
format	TEXT	e.g. 'gguf'
status	TEXT	ready deploying deployed
created_at	TIMESTAMPTZ	

TABLE: datasets*RLS: SELECT/INSERT/UPDATE/DELETE own | idx_datasets_user*

Column	Type	Constraint / Notes
id	UUID	PK
user_id	UUID	FK → auth.users
project_id	UUID	FK → projects nullable
name	TEXT	
description	TEXT	
rows	INTEGER	
columns	INTEGER	
file_size	TEXT	
format	TEXT	e.g. 'jsonl'
quality_score	NUMERIC	0.0 – 1.0
created_at	TIMESTAMPTZ	

TABLE: deployed_endpoints*RLS: SELECT/INSERT/UPDATE/DELETE own | idx_deployed_endpoints_user*

Column	Type	Constraint / Notes
id	UUID	PK
user_id	UUID	FK → auth.users
model_id	UUID	FK → trained_models nullable
model_name	TEXT	
project_name	TEXT	
endpoint_url	TEXT	
status	TEXT	default 'inactive'
requests_per_min	NUMERIC	default 0
avg_latency_ms	NUMERIC	default 0
error_rate	NUMERIC	default 0
uptime	NUMERIC	default 100
rate_limit_per_min	INTEGER	default 60
burst_limit	INTEGER	default 100
created_at	TIMESTAMPTZ	

TABLE: api_keys*RLS: SELECT/INSERT/UPDATE/DELETE own | idx_api_keys_user*

Column	Type	Constraint / Notes
id	UUID	PK
user_id	UUID	FK → auth.users
name	TEXT	
key_prefix	TEXT	e.g. sk-slm-prod
key_suffix	TEXT	Last 4 chars
status	TEXT	default 'active' – active revoked
last_used_at	TIMESTAMPTZ	nullable
created_at	TIMESTAMPTZ	

TABLE: api_call_events*RLS: SELECT/INSERT/DELETE own | idx_api_call_events_user_created*

Column	Type	Constraint / Notes
id	UUID	PK
user_id	UUID	FK → auth.users
endpoint	TEXT	Called endpoint URL
status_code	INTEGER	HTTP status code
latency_ms	INTEGER	Latency in ms
created_at	TIMESTAMPTZ	